Group 04

MEDI-CINE
Software Development Plan (Small Project)
Version 1.0

Revision History

Date	Version	Description	Author
03/Oct/26	0.1	Initial project proposal and scope baseline	Group 04
13/Oct/26	1.0	Strengthened estimates, schedule controls, deliverable acceptance, and reporting metrics	Group 04

Table of Contents

1. Introduction 3

1.1 Purpose 3

1.2 Scope 3

1.3 Overview 3

2. Project Overview 3

2.1 Project Purpose, Scope, and Objectives 3

2.2 Assumptions and Constraints 3

2.3 Project Deliverables 4

3. Project Organization 4

3.1 Organizational Structure 4

3.2 Roles and Responsibilities 4

4. Management Process 5

4.1 Project Estimates 5

4.2 Project Plan 5

4.2.1 Phase Plan 5

4.2.2 Sprint Objectives and Build Plan 5

4.2.3 Release and Sprint Schedule 6

4.2.4 Project Schedule 6

4.2.5 Project Resourcing 6

4.3 Project Monitoring and Control 6

4.3.1 Requirements Management 6

4.3.2 Reporting and Measurement 6

4.3.3 Risk Management 6

4.3.4 Configuration Management 7

Software Development Plan (Small Project)

1. Introduction

1.1 Purpose

This Software Development Plan defines the approach, responsibilities, schedule, controls, and delivery criteria for MEDI-CINE, an online cinema management and ticketing platform. It is used by the project team to coordinate the work, manage scope and risks, and verify that the product is delivered as a responsive and usable web application.

The following people use the Software Development Plan:

- The **project manager** uses it to plan the project schedule and resource needs, and to track progress against the schedule.
- **Project team members** use it to understand what they need to do, when they need to do it, and what other activities they are dependent upon.

1.2 Scope

The plan covers analysis, UX design, implementation, integration, testing, deployment, and handover for the MEDI-CINE system. The product includes an administration dashboard for cinema staff and a responsive web experience for customers. PWA support and an AI movie recommendation assistant are optional extensions, not core PA1 commitments. The initial scope is approximately 15-20 screens.

1.3 Overview

The plan is organized into project overview, organization, management process, estimates, schedule, resourcing, monitoring, reporting, risk management, and configuration management. The project is based on the supplied proposal and its requirements are refined through team review and sprint planning.

Project Overview — provides a description of the project's purpose, scope, and objectives. It also defines the deliverables that the project is expected to deliver.

Project Organization — describes the organizational structure of the project team.

2. Project Overview

2.1 Project Purpose, Scope, and Objectives

MEDI-CINE is a platform for managing cinemas and selling movie tickets online. Administrators and staff can configure cinemas, screening rooms, seating layouts, ticket types, prices, movies, schedules, promotions, and revenue reports. Customers can browse movies and showtimes, select seats in real time, pay online, receive QR e-tickets, review booking history, and rate movies.

The primary objectives are to reduce queueing at ticket counters, prevent schedule conflicts, support flexible pricing, improve seat utilization, and provide a convenient end-to-end booking experience.

2.2 Assumptions and Constraints

The project is delivered by a five-person student team. The first release targets a responsive web application and a desktop browser dashboard; it does not require a native mobile application. The team assumes access to a payment sandbox, email or notification service, representative cinema data, and test devices. Schedule and scope are constrained by academic milestones, limited staffing, integration dependencies, and the need to protect real-time seat availability during booking.

2.3 Project Deliverables

The project will deliver a requirements baseline, UX flows and approximately 15-20 responsive screens, a working customer web application, an admin/staff dashboard, database and API components, payment integration in a sandbox environment, QR e-ticket generation, test cases and results, deployment documentation, and a final demonstration package.

The functional baseline includes 14 core capabilities: authentication and authorization; movie catalogue management; cinema room and seat management; showtime scheduling; ticket pricing; movie and showtime discovery; seat selection and temporary holds; booking and payment; QR e-ticket and check-in; booking history; dashboard and reports; reviews and ratings; promotions and discount codes; and notifications and reminders. AI movie/showtime assistance and PWA support are optional planned extensions and are excluded from the core release commitment.

Deliverable acceptance and target milestones: the requirements baseline and prioritized backlog are reviewed by the end of Sprint 1; UX flows, database design, and API contracts are reviewed by the end of Sprint 2; the end-to-end booking workflow is demonstrated by the end of Sprint 3; the integrated beta and its test results are reviewed by the end of Sprint 4; and the final build, test report, deployment instructions, and demonstration package are completed by the end of Sprint 5. Calendar dates are maintained in the project schedule and confirmed against the course calendar.

3. Project Organization

3.1 Organizational Structure

The team uses a small-project structure led by a team leader. Development work is organized around full-stack, backend, frontend/AI, data, DevOps, and testing responsibilities, with peer review and shared decisions for architecture and scope. Stakeholder feedback is collected at sprint demonstrations.

3.2 Roles and Responsibilities

Person	Role
Trần Tấn Phát (24120118)	Team leader / Full-stack / DevOps
Huỳnh Vĩnh An (24120162)	Full-stack / Data Engineer
Vân Hoàng Nhật (24120209)	Secretary / AI Engineer / Frontend
Lê Hoàng Nam (24120385)	Backend / Tester
Nguyễn Hoàng Nam (24120097)	Backend / Tester

The roles and assignments are listed in the organization table. Every member participates in backlog refinement, code review, integration testing, and project demonstrations. The team leader coordinates scope and delivery; the secretary maintains data and project records; engineering owners implement their assigned services; and testers maintain test coverage and defect status.

For each sprint backlog item, the team records one primary owner and any supporting reviewer. The team leader coordinates schedule, scope, and release decisions; the data engineer coordinates data integrity with backend owners; feature owners implement and document their changes; and testers verify acceptance criteria and maintain defect status. These assignments are confirmed by the team during sprint planning.

4. Management Process

4.1 Project Estimates

The initial estimate is based on the agreed scope of 14 core capabilities, a five-member student team, three development phases, and five sprints. It covers requirements analysis, UX design, implementation, integration, testing, deployment preparation, and documentation.

Schedule estimate: the project follows the academic calendar and the five-sprint baseline in Section 4.2.3. The team maintains planned start/end dates and review dates in the sprint calendar. Exact calendar dates must be confirmed against the course schedule before the baseline is approved.

Effort estimate: before each sprint, team members estimate the effort for assigned backlog items in hours or person-days. The team leader consolidates the estimates and checks them against member availability, task dependencies, and the remaining academic schedule. The total estimate is updated when the backlog is refined; estimates must be based on task-level estimates rather than an unsupported fixed number.

Cost estimate: the project is developed by a five-member student team. The team will prioritize free or student-accessible tools and a payment sandbox where available. Any expected hosting, domain, or third-party service expenses must be recorded in VND before the budget baseline is approved. If no paid services are required, the team records zero planned cash expenditure and documents that assumption.

Estimation basis and re-estimation: estimates use the Vision Document, the prioritized backlog, the team's available skills, technical dependencies, and the academic calendar. Effort, schedule, and cost are reviewed at the end of every sprint, after a major dependency changes, or when a scope change or significant defect affects planned work. Revisions and their reasons are recorded by the team leader.

4.2 Project Plan

The project is organized into three phases and five sprints. Each sprint produces a demonstrable increment. Work is prioritized around the booking path and the administrative data needed to operate a cinema, followed by integration, testing, and release preparation.

4.2.1 Phase Plan

Phase 1 - Inception: confirm users, problem statement, scope, requirements, backlog, team responsibilities, and initial architecture.

Phase 2 - Elaboration: refine UX flows, responsive screens, data model, API contracts, risks, and test approach.

Phase 3 - Construction: implement, integrate, test, and demonstrate the 14 core capabilities covering administration, movie/showtime browsing, seat selection, booking, payment, QR ticketing, reporting, reviews, promotions, and notifications. Optional AI and PWA extensions may be explored only after the core booking workflow is stable.

4.2.2 Sprint Objectives and Build Plan

Build 0.1 is created at the end of Sprint 1 as an internal prototype covering the requirements baseline, repository, architecture, data model, authentication, and basic navigation.

Build 0.2 is created at the end of Sprint 2 after the first administration, movie, room, seat, and showtime capabilities are integrated.

Build 0.3 is created at the end of Sprint 3 after the first end-to-end browsing, seat selection, booking, and payment flow is demonstrated.

Build 0.4 is created at the end of Sprint 4 after QR tickets, booking history, reporting, reviews, promotions, and notifications are integrated. Optional AI and PWA work is not required for Build 0.4.

Build 1.0 is created at the end of Sprint 5 as the final PA1/academic demonstration build after integration testing, defect correction, documentation, and deployment rehearsal.

4.2.3 Release and Sprint Schedule

The project follows three phases and five sprints: Sprint 1 in Inception, Sprints 2 and 3 in Elaboration, and Sprints 4 and 5 in Construction. For every sprint, the schedule records planned start and end dates, review date, sprint goal, task owners, dependencies, and release milestone. Sprint dates are confirmed against the academic calendar before the schedule baseline is approved. Build 0.1, 0.2, 0.3, and 0.4 are incremental internal builds; Build 1.0 is the final academic demonstration build. Optional AI and PWA extensions will not delay the delivery of the 14 core capabilities.

4.2.4 Project Schedule

The project schedule is maintained as a sprint calendar with planned start/end dates, milestone dates, task owners, dependencies, and completion status. The team leader reviews progress weekly against the sprint baseline. The main milestones are: requirements and backlog approval (Sprint 1); UX, architecture, database, and API review (Sprint 2); end-to-end booking demonstration (Sprint 3); integrated beta testing (Sprint 4); and final defect triage, deployment rehearsal, documentation, and demonstration (Sprint 5). Each milestone is accepted only when its deliverables are available and the relevant review or test evidence is recorded. When work is delayed, the owner reports the cause and impact, and the team decides whether to resolve a dependency, reassign work, or defer lower-priority scope.

4.2.5 Project Resourcing

The project has five members: Trần Tấn Phát (team leader, full-stack and DevOps), Huỳnh Vĩnh An (full-stack and data engineer), Vân Hoàng Nhật (secretary, AI engineer and frontend), Lê Hoàng Nam (backend and tester), and Nguyễn Hoàng Nam (backend and tester). The team requires access to a source repository, issue tracker, database environment, payment sandbox, test browsers, and test phones. Short peer sessions cover API contracts, database conventions, deployment, payment callbacks, and QR validation.

Work allocation and capacity control: each backlog item has one directly responsible owner, an estimate, dependencies, and acceptance criteria. The team confirms individual availability during sprint planning and reserves capacity for integration, regression testing, defect correction, and documentation. Feature owners collaborate with testers on test design and verification. If a member is blocked or a task is at risk, the issue is raised during the weekly review so the team can rebalance work.

4.3 Project Monitoring and Control

The team monitors backlog completion, schedule variance, open and closed defects, test-case pass rate, booking-flow success, and integration readiness. Changes are recorded as backlog items and reviewed by the team leader before implementation.

4.3.1 Requirements Management

Requirements are maintained as prioritized backlog items linked to user roles and acceptance criteria. The 14 core capabilities in the Vision Document form the initial product baseline. AI assistance and PWA support remain optional planned extensions. Changes are reviewed by the team leader and affected owners, estimated, prioritized, and demonstrated before acceptance.

4.3.2 Reporting and Measurement

4.3.3 Risk Management

Risks are reviewed weekly and at each sprint boundary. High-impact risks are assigned an owner, a mitigation, and a contingency. The current risk register is included below.

Each risk is assigned an owner during sprint planning. The owner reviews whether the risk is still active, records evidence or warning signs, and reports changes in likelihood or impact. High risks involving duplicate seat bookings and payment integration are checked during relevant integration tests. The team leader coordinates mitigation decisions and escalates any risk that threatens the agreed core scope or final demonstration.

Risk Ranking (High, Medium, Low)	Risk Description and Impact	Mitigation Strategy and/or Contingency Plan
High	Real-time seat conflicts or stale seat state could cause duplicate bookings.	Use temporary seat holds, transactional updates, expiry handling, and concurrency tests.
High	Payment callback or sandbox behavior may delay end-to-end booking.	Integrate early with a sandbox, isolate payment adapters, and provide a test payment path.
Medium	Scope growth from reports, notifications, and AI may affect the release date.	Prioritize the core booking path and treat AI recommendations as an extension.
Medium	Limited team time may reduce test coverage and deployment readiness.	Run weekly triage, automate repeatable tests, and freeze the release scope before hardening.

4.3.4 Configuration Management

The project uses a shared Git repository as the configuration-management repository. The main branch contains the reviewed baseline; feature branches are created for isolated work and merged only after peer review and relevant tests pass.

Configuration items include source code, database scripts and seed data, API contracts, UI assets, test cases/results, requirements and Vision updates, the Software Development Plan, weekly reports, deployment settings, and demonstration documentation. Each accepted sprint build is tagged as Build 0.1, 0.2, 0.3, 0.4, or 1.0.

Changes to scope, requirements, architecture, interfaces, or release content are recorded as backlog/change items, reviewed by the team leader and affected owners, assessed for impact, and accepted during sprint planning or review. The repository and documents are backed up through the team workspace, and release baselines are preserved so the team can identify the exact version used for testing and demonstration.